Web应用开发C

成都信息工程大學
Chengdu University of Information Technology

学校：成都信息工程大学

学院：软件工程学院

专业：数据科学与大数据技术

授课教师：杨与广

邮箱：yyg@cuit.edu.cn

群名称:数科221-Web应用开发C
群　号:870648816

第五章 Flask-安装及基本结构 总结

- 安装　　环境配置流程
- 应用的基本结构　　Hello World、初始化、路由、视图函数、request、response、运行、MTV

第六章 Flask-模板



成都信息工程大學
Chengdu University of Information Technology

- 模板
- Jinja2
- 链接和静态文件

第六章 Flask-模板

- 模板
- Jinja2
- 链接和静态文件

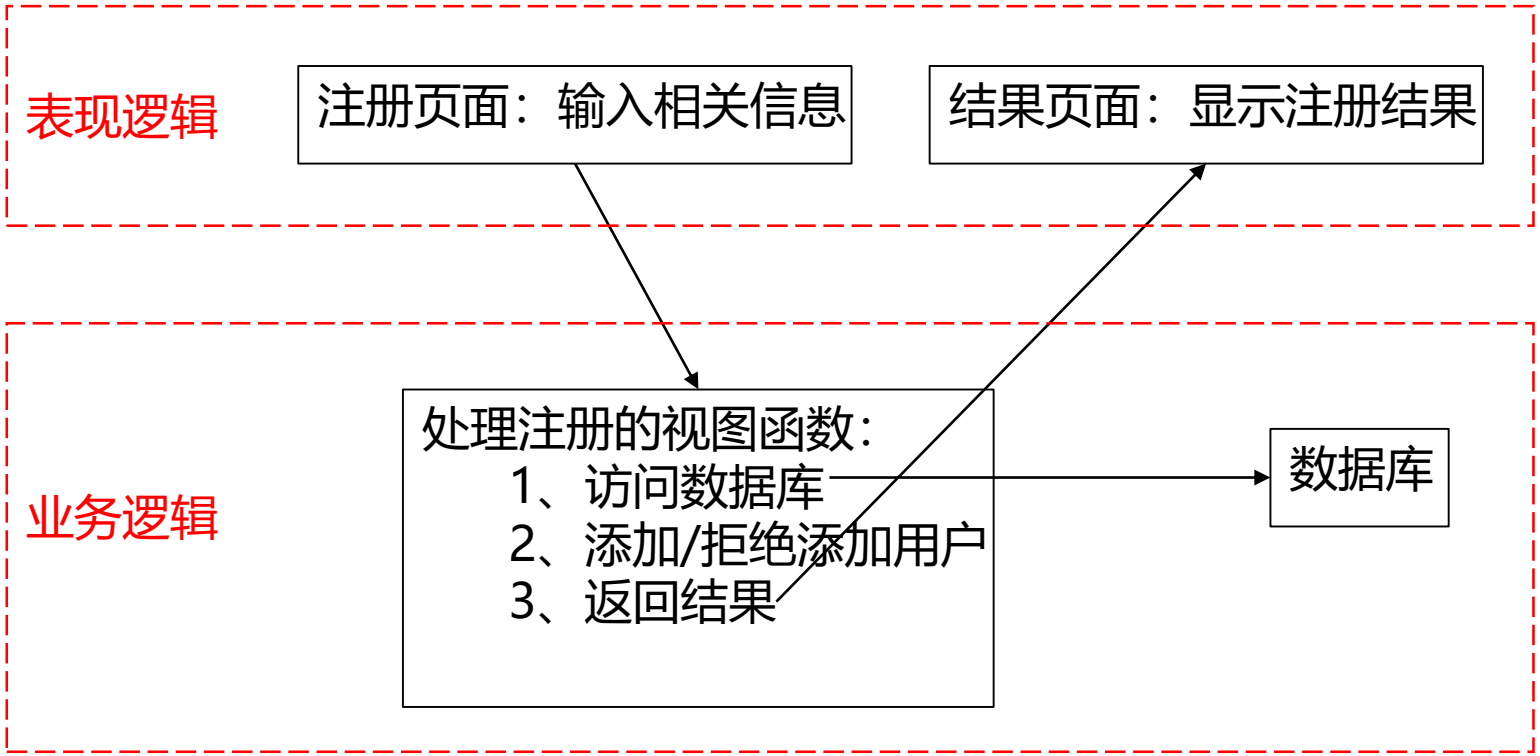
模板

MTV

- 在Web开发中，架构模式是一种组织和管理代码的方式，它能够使我们的应用程序更易于理解、扩展和维护。MTV架构是一种常见的Web开发架构，它将应用程序分为三个主要组件：模型(Model)、模板(Template)和视图(View)。
  - ✓ 模型是MTV架构的核心组件，它负责定义应用程序的数据结构和业务逻辑
  - ✓ 模板是MTV架构的表示层组件，它负责生成应用程序的用户界面
  - ✓ 视图是MTV架构的控制层组件，它负责处理用户请求，并根据请求的类型和参数来决定如何处理请求

模板

- 为什么
  - 实现业务逻辑和表现逻辑分离，方便分工
  - 提高应用的可维护性



模板

模板的主要内容

- 模板引擎，为使界面与业务数据分离，可以生成特定格式的文档，Jinja2->HTML
- 模板渲染，将变化的内容表现出来的过程
- 模板变量，将变化的内容在模板中占位
- 模板过滤器，对数据的一些处理
- 模板控制结构，用于改变模板的渲染流程
- 模板继承，模板的组织方式

第六章 Flask-模板

- 模板
- Jinja2
- 链接和静态文件

Jinja2

https://doc.yonyoucloud.com/doc/jinja2-docs-cn/index.html

简介

- Jinja2 是一个现代的，设计者友好的，仿照 Django 模板的 Python 模板语言。它速度快，被广泛使用，并且提供了可选的沙箱模板执行环境保证安全
- Flask 使用 Jinja2 作为默认模板引擎。你完全可以使用其它模板引擎。但是不管你使用哪种模板引擎，都必须安装 Jinja2 。因为使用 Jinja2 可以让 Flask 使用更多依赖于这个模板引擎的扩展。

特性

- 强大的 HTML 自动转义系统保护系统免受 XSS
- 模板继承
- 及时编译最优的 python 代码
- 易于调试。异常的行数直接指向模板中的对应行。

Jinja2

模板渲染

- flask 使用render_template(模板名，key=value，key=value)完成模板渲染
- render_template()一般包含*两个*参数，第一个模板文件名，另一个是参数对
- 参数对中的key表示在模板中的占位符，用{{key}}在模板中表示，value表示对应key的值，渲染后由value替换{{key}}，从而完成视图函数与模板的值传递

```
@app.route('/rendertemplate')
def requestData():
    return render_template('renderTemplate.html')


@app.route('/rendertemplate2')
def renderTemplate2():
    return render_template('renderTemplate.html',name='zhansan',age='18')
```

5、模板渲染.py × renderTemplate.html ×

```
1   <!DOCTYPE html>
2   <html lang="en">
3   <head>
4       <meta charset="UTF-8">
5       <title>renderTemplate</title>
6   </head>
7   <body>
8   <div> <h1>模板渲染演示</h1> </div>
9   <p>
10      姓名：{{name}} <br>
11      年龄：{{age}}
12  </p>
13  </body>
14  </html>
```

Jinja2

模板变量

- 模板变量支持：所有类型的变量，字符串、整形、浮点、列表、字典或对象等
- 一般的变量在模板中用key占位，渲染时由value替换key
- 模板的列表访问，索引、切片，如：list[0]/list.0/list[:]
- 模板的字典访问，dict.key/dict.get(key)
- 模板的对象访问，对象名.属性

```python
@app.route('/rendertemplate3')
def renderTemplate3():
    name = 'zhansan'
    age = 18
    gender = 'male'
    classmate = ['lisi','wangwu','zhaoliu']
    girlfriend = {'name': 'datiechui', 'age': 15, 'gender': 'female'}
    soulmate = Girl( name: "yuyan", age: 18)
    return render_template( template_name_or_list: 'renderTemplateVar.html',
                           name=name,age=age,gender=gender,classmate=classmate,
                           girlfriend=girlfriend,soulmate=soulmate)
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>renderTemplate</title>
</head>
<body>
 <div> <h1>模板渲染变量演示</h1> </div>
<p>
    姓名：{{name}} <br>
    年龄：{{age}} <br>
    性别：{{gender}} <br>
    同学：{{classmate}}---{{classmate[0]}}---{{classmate[1]}}---{{classmate.2}} <br>
    女朋友：{{girlfriend}}---{{girlfriend.name}}---{{girlfriend.age}}---{{girlfriend.get('gender')}} <br>
    灵魂伴侣：{{soulmate}}---{{soulmate.name}}---{{soulmate.age}}---{{soulmate.gender}} <br>
</p>
</body>
</html>
```

Jinja2

模板过滤器

- 过滤器能够实现对变量的处理
- 本质上是函数，可传递参数/自定义
- 添加在变量名之后用竖线(|)隔开
  {{变量名|过滤器名}} 可链式调用
- 常用的过滤器

列表过滤器	说明
first	第一个
last	最后一个
length	长度
sum	求和，对数值型
sort	排序

字符串过滤器名	说明
safe	禁用转义
capitalize	首字母大写，其他小写
lower	转换为小写
upper	转换为大写
title	每个单词的首字母大写
trim	删除首尾空格
striptags	渲染前删除所有的HTML标签
reverse	反转
format()	格式化字符串

http://doc.yonyoucloud.com/doc/jinja2-docs-cn/templates.html#builtin-filters

1、app.add_template_filter(function,name='过滤器名称 ')
2、@app.template_filter('过滤器名称')

模板控制结构

- 用于改变模板的渲染流程
- 选择：

```
{% if %}
    ……
{% else if %}
    ……
{% else %}
    ……
{% endif %}
```

- 循环：

```
{% for item in items %}
    ……
{% endfor %}
```

- 嵌套：

```
{% for item in items %}
    {% if %}
        ……
    {% else if %}
        ……
    {% else %}
        ……
    {% endif %}
{% endfor %}
```

模板控制结构

if 示例

```
9   <p>
10      {% if name %}
11          姓名: {{name}} <br>
12          年龄: {{age}} <br>
13          性别: {{gender}} <br>
14          同学: {{classmate[0]}}---{{classmate[1]}}---{{classmate.2}} <br>
15      {% else %}
16          Hello Stranger, Plsease Login In
17      {% endif %}
18  </p>
```

```
20  <p>
21      {% if name %}
22          姓名: {{name}} <br>
23          年龄: {{age}} <br>
24          性别: {{gender}} <br>
25          同学:
26              {% for item in classmate %}
27                  {{ item }}
28              {% endfor %}
29              <br>
30      {% else %}
31          Hello Stranger, Plsease Login In
32      {% endif %}
33  </p>
```

Jinja2

模板控制结构

- for 示例
- 可以使用loop变量来实现循环控制
  - ✓ loop.index　　循环序号从1开始
  - ✓ loop.index0　　循环序号从0开始
  - ✓ loop.length　　循环的数量
  - ✓ loop.last　　是否是最后一个
  - ✓ loop.first　　是否是第一个
  - ✓ loop.revindex0序号倒序从0开始
  - ✓ loop.revindex　序号倒序从1开始

```
<p>
    姓名：{{name}} <br>
    年龄：{{age}} <br>
    性别：{{gender}} <br>
    所有同学：{% for classmate in classmates %}
                <ul>
                    <li>{{ classmate }}</li>
                </ul>
            {% endfor %}
</p>
```

```
<p>
    姓名：{{name}} <br>
    年龄：{{age}} <br>
    性别：{{gender}} <br>
    部分同学：{% for classmate in classmates %}
                <ul>
                    {% if loop.last %}
                        <li  style="color: red" >{{ classmate }}</li>
                    {% else %}
                        <li >{{ classmate }}</li>
                    {% endif %}
                </ul>
            {% endfor %}
</p>
```

```
<p>
    姓名：{{name}} <br>
    年龄：{{age}} <br>
    性别：{{gender}} <br>
    部分同学：{% for classmate in classmates %}
                <ul>
                    <li {% if loop.last %} style="color: red" {% endif %}>{{ classmate }}</li>
                </ul>
            {% endfor %}
</p>
```

模板继承

- 可实现模板的复用，将模板中共同的部分提炼出来，避免重复编写
- 实现：
  - ✓ 1、父模板中用{% block 块名 %} {% endblock %}定义变化的部分
  - ✓ 2、子模块中用{% extends “父模块” %}继承父模块，
  - ✓ 3、 用{% block 块名 %} {% endblock %}在其内部写上子模块自己的内容
- 注意
  - ✓ 父模板原内容若需要修改，应放在block定义的中间部分
  - ✓ 子模块需要修改的部分需在父模块中定义好，不然无法生效
  - ✓ 标签必须写在block内部
  - ✓ 一般需要在父模板中 定义好css和 js的block，js的block尽量放在父模板body的底部
  - ✓ 可用{{super()}}调用父模块的内容

Jinja2

https://flask-bootstrap-zh.readthedocs.io/zh/latest/

https://getbootstrap.com/

利用Flask-Bootstrap 集成Bootstrap

- Flask-Bootstrap 是把 Bootstrap 打包进一个扩展，方便使用Bootstrap框架内容
- 实现：
  - ✓ 下载Flask-Bootstrap，
  - ✓ 创建一个Bootstrap对象并与app关联
  - ✓ 利用模板继承的方式构建基于bootstrap/base.html的模板
- Flask-Bootstrap base.html中定义的区块

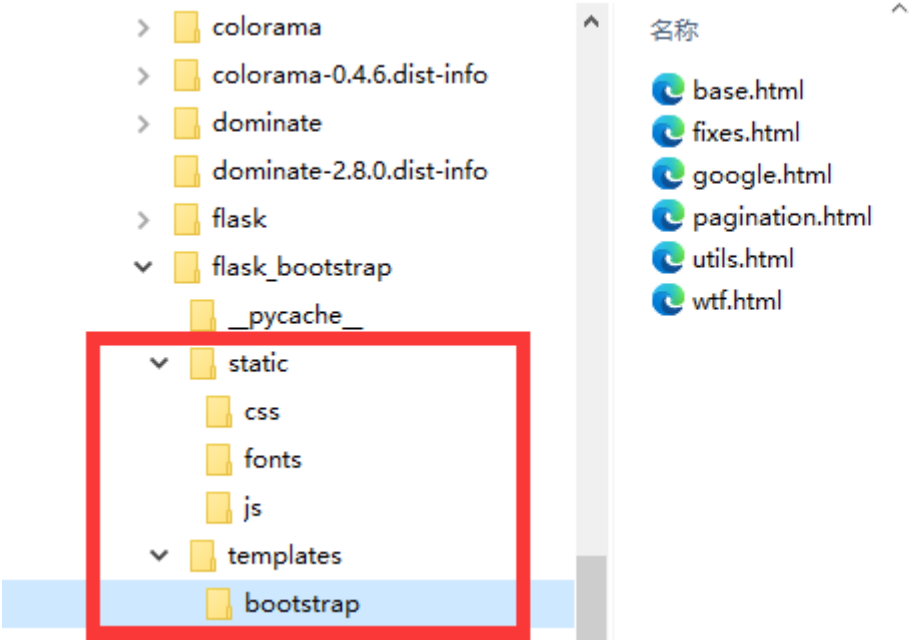
```
pip install flask-bootstrap

from flask_bootstrap import Bootstrap
        ……
bootstrap = Bootstrap(app)
```

doc	整个html文档	styles	CSS声明
html_attribs	html标签的属性	body_attribs	body标签的属性
html	html标签中的内容	body	body标签的内容
head	head标签中的内容	navbar	用户定义的导航栏
title	title标签中的内容	content	用户定义的页面内容
metas	一组meta标签	scripts	文档底部的script声明

Jinja2

利用Flask-Bootstrap 集成Bootstrap




```
{% block doc -%}
<!DOCTYPE html>
<html{% block html_attribs %}{% endblock html_attribs %}>
{%- block html %}
  <head>
    {%- block head %}
    <title>{% block title %}{{title|default}}{% endblock title %}</title>

    {%- block metas %}
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    {%- endblock metas %}

    {%- block styles %}
    <!-- Bootstrap -->
    <link href="{{bootstrap_find_resource('css/bootstrap.css', cdn='bootstrap')}}" rel="stylesheet">
    {%- endblock styles %}
    {%- endblock head %}
  </head>
  <body{% block body_attribs %}{% endblock body_attribs %}>
    {% block body -%}
    {% block navbar %}
    {%- endblock navbar %}
    {% block content -%}
    {%- endblock content %}

    {% block scripts %}
    <script src="{{bootstrap_find_resource('jquery.js', cdn='jquery')}}"></script>
    <script src="{{bootstrap_find_resource('js/bootstrap.js', cdn='bootstrap')}}"></script>
    {%- endblock scripts %}
    {%- endblock body %}
  </body>
{%- endblock html %}
</html>
{% endblock doc -%}
```

Jinja2

include 和 宏

- include：当部分页面包含共同的部分时，可以将这些共同的部分单独做成一个模板，然后利用include引入到模板中。
  - ✓ 首先定义通用部分模板：common.html
  - ✓ 然后在使用通用部分的模板中：{% include ‘common.html’ %}
  - ✓ include将原网页全包含到新模板中
- 宏：可以当做模板中的函数，可以返回一个HTML字符串，实现代码复用，避免代码冗余
  - ✓ 利用{% macro 宏名(参数列表) %} 通用的HTML {% endmacro %} 在单独的文件中定义宏，一般将宏定义在单独的文件中，macro.html
  - ✓ 在需要用到宏的模板中利用 {% import ‘macro.html’ as macro %} {{宏名(参数列表)}}

第六章 Flask-模板

- 模板
- Jinja2
- 链接和静态文件

链接和静态文件

- 链接
  - 任何具有多路由的应用都需要可以连接不同页面的链接，例如导航栏；
  - 为了在模板或者定义路由时，应对动态URL问题，需要用到url_for()函数，返回资源的URL
  - 实现：
    - ✓ from flask import url_for
    - ✓ url_for(endpoint，关键字参数列表)
    - ✓ 返回一个动态的URL

url_for('User', name='lisi', age=20,gender="male")

/User/lisi?age=20&gender=male

  - url_for()函数也可以定位应用的静态文件(HTML代码引用的图像、声音、js源码文件和css等)
    - ✓ 静态文件一般放在static文件夹内
    - ✓ 用法：url_for('static'，filename='静态文件地址')

第六章 Flask-模板 总结

- 模板　　MTV、模板
- Jinja2　　简介、渲染、变量、过滤器、控制结构、继承、集成Bootstrap、include和宏
- 链接和静态文件　　链接、静态文件